

Code review — design-patterns-csharp

Runde de review pe proiectul tău, cea mai nouă sus.

Ultima rundă: 2026-09-23 (runda 6) · **Commit:** `c6df8dc` „testare” (munca reală e în `dc17712`) ·

Build:  `0 Error(s)`, 3 warning-uri · **Rulare:**  **programul moare la ex9 cu**

`NullReferenceException` — ex10 și ex11 nu se execută niciodată

Runda 6 — Factory, Decorator și un NRE care ascunde tot (c6df8dc)

Ai scris două exerciții noi — ex10 (Factory) și ex11 (Decorator) — și le-ai pushat pe amândouă. Problema e că **n-ai apucat să le vezi rulând**, și nu din vina lor: programul moare înainte să ajungă la ele.

Ca să le pot evalua, le-am rulat izolat, într-o copie din scratchpad. Codul tău n-a fost atins.

Ce s-a închis din runda 5

Constatare	Stare
B1 — ex5 nu afișa verdictele	⚠ pe jumătate — vezi M2
B2 — ex9 , reintrarea	⚠ mutat, nu rezolvat — vezi B3

Ambele au aceeași formă ca data trecută: **fix-ul aplicat pe jumătate**. Runda 5 spunea, despre exact acest lucru: „o greșeală de gândire îmbrăcată diferit: un fix mutat pe jumătate”. S-a repetat, dar mai subtil — de data asta ai scris soluția corectă și ai pus-o în locul greșit.

● Critice

B1 — `NullReferenceException` la ex9: programul moare, ex10 și ex11 nu rulează niciodată

ex9/Testare9.cs:17–19 , ex9/Models/Licitatie.cs:32–38 , ex9/Models/AutoLicitator.cs:16

CUM E ACUM — `dotnet run` , ultimele linii:

```
[SPECTATOR] Pret curent: 10
Unhandled exception. System.NullReferenceException
  at AutoLicitator.OfertaNoua(Decimal) in AutoLicitator.cs:line 16
  at Licitatie..ctor(Decimal, IParticipant[]) in Licitatie.cs:line 37
  at Testare9..ctor() in Testare9.cs:line 17
  at Program.Main() in Program.cs:line 25
```

`Program.cs:25` e `Testare9`. Liniile 26 și 27 — `Testare10` și `Testare11`, adică toată munca rundei ăsteia — nu se mai execută. Un cititor al repo-ului tău rulează proiectul și **nu vede Factory-ul și Decoratorul deloc**. La olimpiadă, asta înseamnă zero puncte pe două cerințe rezolvate.

DE CE se întâmplă:

```
Licitatie licitatie = new(10, participanti); // ← difuzează CHIAR ACUM
auto1.Licitatie = licitatie;                // ← legătura se face abia AICI
auto2.Licitatie = licitatie;
```

Constructorul `Licitatie` nu doar inițializează — el **difuzează**: la `Licitatie.cs:37` cheamă `participanti[i].OfertaNoua(PretCurent)` pentru fiecare participant. Dar în momentul ăla, `auto1.Licitatie` e încă `null`, pentru că atribuirea din `Testare9.cs:18` n-a fost încă executată. Primul `AutoLicitator` intră în `OfertaNoua`, ajunge la `if (pretCurent != Licitatie.PretCurent)` și citește un câmp al unui obiect care nu există.

Spectatorul trece — el nu se uită la `Licitatie`. De-aia vezi `[SPECTATOR] Pret curent: 10` și abia apoi excepția: ordinea din array e `Spectator, auto1, auto2`.

Rădăcina e o **dependență circulară**: licitația are nevoie de participanți ca s-o construiască, participanții au nevoie de licitație ca să răspundă. Ai rezolvat-o legându-i după construcție — corect ca idee — dar constructorul nu așteaptă până atunci.

Un constructor care face **muncă**, nu doar inițializare, e capcana. Când constructorul se termină, obiectul e gata; dar în timpul lui, lumea din jur poate fi încă pe jumătate construită.

FIX: scoate difuzarea din constructor. Constructorul doar setează `PretCurent` și `participanti`; deschiderea licitației devine o metodă separată, chemată după ce legăturile există:

```
Licitatie licitatie = new(10, participanti);
auto1.Licitatie = licitatie;
auto2.Licitatie = licitatie;
licitatie.Deschide(); // ← abia acum se difuzează
```

B2 — ex11: niciunul dintre cei trei decoratori nu delegă mai departe

`ex11/Models/CuJurnal.cs:25` , `ex11/Models/CuTrunchiere.cs:38` , `ex11/Models/CuPrefix.cs:33`

CUM E ACUM: fiecare decorator ține o referință `interior` , o folosește ca să-și compună `Nume` ... și apoi scrie singur cu `Console.WriteLine` . Niciunul nu cheamă `interior.Trimite(mesaj)` .

Rulat izolat, ex11 scoate:

```
[EMAIL] Lorem Ipsum is simply...      ← email.Trimite() direct
Email [URGENT] : Lorem Ipsum is...    ← prefix.Trimite() – unde e [EMAIL]?
```

Prima linie are `[EMAIL]` , a doua nu. Adică `NotificatorEmail.Trimite` **nu s-a executat** când ai trecut prin `CuPrefix` . Decoratorul l-a înlocuit, nu l-a împachetat.

A doua dovadă, și mai limpede — liniile 29 și 30 din `Testare11` :

```
[JURNAL] catre SMS + trunchiere: ...test test test test test test test  ← COMPLET
SMS + jurnal + trunchiere: ...test test test test tes                    ← TRUNCHIAT
```

`new CuJurnal(trunchi)` ar trebui să logheze **și apoi** să trimită prin `trunchi` , care trunchiază. Jurnalul are textul întreg, pentru că `CuJurnal` s-a oprit la el însuși. În ambele linii se execută exact **un** decorator: cel de la exterior. Lanțul nu există.

DE CE contează, dincolo de output: ăsta e chiar mecanismul pattern-ului. Un Decorator implementează aceeași interfață ca obiectul decorat și ține o referință la el, tocmai ca să poată face *ceva înainte* → **cheamă interiorul** → *ceva după*. Dacă scoți apelul din mijloc, îți rămâne o clasă care implementează interfața și ignoră ce a primit — adică o **implementare alternativă**, nu un decorator. De-aia poți împacheta oricâte: fiecare adaugă un strat și pasează mai departe. La tine, oricâte ai împacheta, se execută doar ultimul înveliș.

Diferența se vede și în `Nume` : acolo **ai delegat** (`interior.Nume + " + jurnal"`), și de aia numele compuse iese corect — „SMS + jurnal + trunchiere”. Ai făcut lanțul pentru nume și nu pentru comportament.

FIX: în fiecare `Trimite` , după partea proprie, cheamă interiorul:

```
public void Trimite(string mesaj)
{
    Console.WriteLine($"[JURNAL] catre {interior.Nume}: {mesaj}");
    interior.Trimite(mesaj);          // ← asta lipsește
}
```

La `CuTrunchiere`, trunchierea se face **înainte** de delegare, iar la `CuPrefix` prefixul se lipește de mesaj și se pasează mai departe — nu se afișează pe loc.

B3 — B2 din runda 5: ai scris exact soluția cerută, dar în locul greșit

`ex9/Models/Licitatie.cs:21-46` (constructor) vs `:49-61` (`Liciteaza`)

Runda 5 îți cerea să scoți reintrarea din `Liciteaza`, cu tiparul cozii: „primești oferta, o pui într-o coadă și, dacă nu ești deja în bucla de procesare, o consumi într-un `while` la nivelul de sus”.

Ai implementat exact asta — `difuzez`, `ofertaInAsteptare`, `areOfertaInAsteptare`, bucla `do...while`. Corect, până la ultima literă.

Doar că e **în constructor**. Metoda `Liciteaza`, adică exact cea pe care o cheamă `AutoLicitator` la linia 21, a rămas neatinsă:

```
public void Liciteaza(decimal suma)
{
    PretCurent = suma;
    for (int i = 0; i < participanti.Length; i++)
        participanti[i].OfertaNoua(suma);    // ← difuzare directă, fără coadă
}
```

Constructorul se cheamă o singură dată și nu e reintrat niciodată. `Liciteaza` se cheamă din interiorul difuzării — acolo era problema. Deci protecția stă fix în locul care n-avea nevoie de ea.

N-am putut să-ți arăt stack overflow-ul de data asta: cu `pretMaxim = 1_000_000` scenariul moare mai devreme, la NRE-ul de la B1. Bug-ul vechi e **acoperit** de unul nou, nu rezolvat.

FIX: mută tot blocul `difuzez` / coadă / `do...while` din constructor în `Liciteaza`. Constructorul rămâne cu atât: validări, `PretCurent = pret`, `this.participanti = participanti`.

● Importante

M1 — `Int32.Parse` reapare, în cod nou

`ex10/Models/FabricaAdmin.cs:9`

```
return new Admin(campuri[1], campuri[2], Int32.Parse(campuri[3]));
```

Am căutat în tot proiectul: **e singura apariție**. Toate cele de dinainte au fost curățate — inclusiv a patra, pe care n-o semnalasem și pe care ai găsit-o singur în runda 5. Aia a fost prima dată când ai aplicat regula, nu linia.

Și totuși iat-o din nou, într-un fișier scris de la zero trei zile mai târziu. Asta spune că regula a fost aplicată ca **operațiune de curățenie** — ai trecut prin cod și ai înlocuit — nu ca reflex la scris. Diferența se vede exact în fișiere noi.

`FabricaAdmin` primește câmpuri dintr-un CSV. Un CSV stricat (`ADMIN, Ion, Pop, zece_mii`) oprește programul cu `FormatException`. La olimpiadă, unde datele vin din fișier, asta e diferența între „am tratat intrarea greșită” și „mi-a crăpat aplicația la evaluare”.

M2 — `ex5` : verdictele se văd, dar toate pe un rând

`ex5/Testare5.cs:14, :18, :22`

B1 din runda 5 e închis pe jumătate: **citești** acum rezultatul lui `Verifica` — asta era esențialul. Dar ai folosit `Console.Write` în loc de `WriteLine` :

```
Simpla: TrueMedie: FalsePuternica: FalsePuternica: False
```

Patru verdicte lipite. Iar linia 24 testează a doua oară `puternica`, cu **aceeași** parolă slabă. Intenția se vede la linia 10: `parolaPuternica = "Abc123!x"` — declarată și niciodată folosită (warning `CS0219`). Voi ai să arăți că politica puternică trece cu o parolă puternică; ai pasat-o pe cea slabă.

M3 — `ex10` : `Trim()` doar pe tip, nu pe valori

`ex10/Models/IncarcatorUtilizatori.cs:26`, `ex10/Testare10.cs:12`

```
Admin:  Scarlat,  Stefan, salariu: 10000
```

Spații duble. Linia de intrare e `"ADMIN, Stefan, Scarlat, 10000"` — cu spațiu după fiecare virgulă, spre deosebire de celelalte trei. Tu faci `campuri[0].Trim()` doar pentru tip, ca să potrivești fabrica; valorile ajung la constructor cu spațiul lipit.

Salariul scapă din întâmplare: `Int32.Parse` ignoră spațiile albe de la margini. Textele, nu.

Un CSV real are spații după virgulă mai des decât n-are. `linii[i].Split(',')` urmat de un `Trim()` pe fiecare câmp rezolvă tot, dintr-un loc.

Cleanups

C1 — ex10 : bucla de căutare a fabricii nu se oprește când a găsit. `IncarcatorUtilizatori.cs:28-34` — după ce `fabrici[j].Tip == tip` se potrivește și obiectul e creat, bucla continuă prin restul fabricilor. Cu 3 fabrici nu contează; e despre reflex. `break;` după atribuire.

C2 — ex9 : două warning-uri CS8618 . `Licitatie.cs:12` (`participanti`) și `AutoLicitator.cs:9` (`Licitatie`) — compilatorul îți spune că un câmp non-nullable poate rămâne `null` după constructor. Exact ce s-a întâmplat la B1. Warning-ul a fost acolo tot timpul.

C3 — ex5 : parolaPuternica declarată și nefolosită (`CS0219`) — vezi M2.

Before / After (doar criticele)

B1 — difuzarea din constructor

Acum (<code>Licitatie.cs:28-46</code>)	Cum trebuie
<pre>difuzez = true; PretCurent = pret; this.participanti = participanti; do { ... } while (areOfertaInAsteptare); difuzez = false;</pre>	<pre>PretCurent = pret; this.participanti = participanti; // metoda separata: public void Deschide() { NotificaToti(PretCurent); }</pre>

B2 — decoratorul care deleagă

Acum (CuJurnal.cs:19–26)	Cum trebuie
<pre>public void Trimite(string mesaj) { Console.WriteLine(\$"[JURNAL] ..."); }</pre>	<pre>public void Trimite(string mesaj) { Console.WriteLine(\$"[JURNAL] ..."); interior.Trimite(mesaj); }</pre>

B3 — unde stă protecția de reintrare

Acum	Cum trebuie
constructor: coadă + <code>do...while</code> <code>Liciteaza</code> : <code>for</code> simplu	constructor: doar validări + atribuiri <code>Liciteaza</code> : coadă + <code>do...while</code>

Q&A

- La ex11, `Nume` iese corect compus („SMS + jurnal + trunchiere”), dar comportamentul nu se compune. Uită-te la cele două: ce face `Nume` și nu face `Trimite` ? De ce compilatorul n-a avut nimic de spus?
- La ex9, warning-ul `CS8618` era afișat la fiecare build, cu mult înainte de a rula programul. Ce-ți spunea el, mai exact — și de ce nu putea compilatorul să-l transforme în eroare?
- Ai pus protecția de reintrare în constructor. Constructorul `Licitatie` se poate reintra vreodată — adică poate ajunge programul să-l execute a doua oară în timp ce prima execuție e încă în curs? Dar `Liciteaza` ?

Runda 5 — fix-urile rundei 4 (00f0adf)

Runda asta e cea mai bună de până acum: **20 din 22 de constatări închise**, inclusiv toate cele 4 critice și blocantul de build. Și, important, **fără nicio regresie în codul de model** — ceea ce runda 2 și runda 4 n-au reușit.

Ce a rămas sunt exact două lucruri, și amândouă sunt aceeași greșeală de gândire îmbrăcată diferit: **un fix mutat pe jumătate**. La ex5 ai scos afișarea din strategie, dar n-ai pus-o înapoi la apelant. La ex9 ai reparat simptomul reintrării (prețul învechit), dar nu și reintrarea în sine.

Rezolvate din runda 4

Constatare	Verificat prin
B0 — proiectul nu compila (9 erori)	 <code>dotnet build</code> → <code>Build succeeded. 0 Error(s)</code>
B1 — <code>Aboneaza</code> ieșea din array	 <code>Canal.cs:16</code> — bucla merge după <code>abonati.Length</code> . Rulat izolat cu 3 abonați: toate 3 notificate
B2 — <code>Dezaboneaza</code> aloca după numărul greșit	 <code>Canal.cs:31</code> numeri cine rămâne , <code>:36</code> parcurgi sursa , indici separați <code>i / cnt</code> . Rulat izolat: <code>[a,b,c]</code> minus <code>b</code> → rămân <code>a</code> și <code>c</code> ; dezabonare pe cineva neabonat → rămân toți 3. Exact cele două cazuri din Q1
B3 — prețuri învechite difuzate	 <code>AutoLicitator.cs:16</code> — <code>if (pretCurent != Licitatie.PretCurent) return;</code> . <code>Testare9</code> urcă 15→110 și se oprește singură, fără excepție
B4 — <code><=</code> la <code>pretMaxim</code>	 <code>AutoLicitator.cs:18</code> — <code>if (pretCurent >= pretMaxim) return;</code>
M1 — <code>Verifica</code> întorcea <code>void</code>	 <code>structural (CampParola.cs:25</code> → <code>bool</code>) — dar vezi B1 de mai jos
M2 — politicile afișau din interiorul validării	 toate trei politicile sunt acum funcții pure: intră <code>string</code> , iese <code>bool</code> , zero <code>Console</code>
M3 — bucla de notificare duplicată în <code>Cont</code>	 <code>Cont.cs:15–21</code> — <code>NotificaObservatori()</code> extras, chemat din ambele
M4 — null-check inconsistent	 parțial — cele 4 locuri citate  , restul nu. Vezi M1
M5 — <code>AutoLicitator.Licitatie</code> publică, setată după construcție	 NEATINS . Vezi M2

Constatare	Verificat prin
M6 — <code>Raport.Date</code> proprietate privată	✓ <code>Raport.cs:6</code> — <code>private readonly string[] date</code>
M7 — <code>Raport</code> nu valida <code>strategie</code>	✓ <code>Raport.cs:15-18</code> + <code>SchimbaFormat</code> la <code>:26</code>
C1 — <code>using</code> nefolosit în <code>Licitatie</code>	✓ scos
C2 — lanț <code>else if</code> în <code>PoliticaPuternica</code>	✓ trei <code>if</code> independente
C3 — <code>Comision</code> care ținea bonusul	✓ <code>ComisionCuBonus.cs:7</code> — <code>Bonus</code>
C4 — <code><=</code> vs <code><</code> la <code>AlertaSoldMic</code>	✓ <code>AlertaSoldMic.cs:14</code>
C5 — <code>ComisionFix</code> fără validare	✓ <code>ComisionFix.cs:10-13</code>
C6 — <code>ArgumentNullException("new policy")</code>	✓ <code>CampParola.cs:20</code> — <code>nameof(noua)</code>
C7 — separator lipit în statistică	✓ <code>AfisajStatistica.cs:24</code>
C8 — <code>decimal</code> numar contor	✓ <code>int</code> numar
C9 — <code>using System.Text</code> abandonat	✓ scos
C10 — parametrul rămas <code>observatori</code>	✓ <code>StatieMeteo.cs:8</code> — <code>afisaje</code>
C11 — <code>"date"</code> literal	✓ <code>nameof(date)</code>
C12 — fișiere fără linie nouă la final	● NEATINS (11 fișiere). Vezi C1

Merită spus separat: **B2 era bug-ul pe care îl reparaseși corect în ex2 acum trei runde și l-ai reconstruit greșit din memorie în ex7**. Acum `ex2/Models/Comanda.cs:27-49` și `ex7/Models/Canal.cs:25-45` sunt identice ca traseu, cu indici separați în ambele. Ai deschis soluția veche. Asta era toată lecția.

● Critice

B1 — rezultatul lui `Verifica` se pierde: `ex5` nu mai afișează nimic

`ex5/Testare5.cs:15, :20, :25, :28`

```
Console.Write(politica.Nume + ": ");  
campParola.Verifica(parola);           // ← întoarce bool, nimeni nu-l citește
```

`dotnet run` , output real, linia de ex5:

```
Simpla: Medie: Puternica: Puternica: Fix: 50.00
```

Patru etichete, zero verdicte, fără linie nouă la final — de aia prima linie din ex6 („Fix: 50.00”) se lipește de ea. Exercițiul S1 nu mai demonstrează nimic: nu poți vedea că `"abc123"` trece la Simplă și pică la Medie, care era exact ideea lui.

DE CE: M1 și M2 din runda 4 ți-au cerut să muți o responsabilitate — afișarea — din strategie în context. Ai făcut prima jumătate (ai scos-o) și n-ai făcut-o pe a doua (n-ai pus-o unde trebuia). Compilatorul n-a spus nimic pentru că în C# un apel de metodă e o **instrucțiune-expresie** perfect legală:

`campParola.Verifica(parola);` compilează la fel de bine ca `x + 1;` n-ar compila — diferența e că un apel poate avea efecte secundare, deci limbajul nu are cum să știe dacă rezultatul te interesează sau nu. Valoarea `true` / `false` se calculează, ajunge pe stivă și e aruncată. Zero erori, zero warning-uri, program mut.

(În .NET există `[MustUseReturnValue]` și analizoare care semnalează asta, dar nu sunt pornite implicit. Plasa de siguranță reală rămâne una singură: **rulezi și te uiți la output**, nu doar la build.)

FIX: `Console.WriteLine(politica.Nume + ": " + campParola.Verifica(parola));`

B2 — **ex9** : reintrarea e încă acolo, doar mascată de cifrele mici din test

`ex9/Models/Licitatie.cs:31-34` + `ex9/Models/AutoLicitator.cs:21`

Garda pe preț învechit (B3, runda 4) e **corectă și necesară** — dar ea repară ce se difuzează, nu cine pe cine cheamă. Reprodus izolat, cu aceleași două clase, singura schimbare fiind `pretMaxim` :

```
var a1 = new AutoLicitator(1_000_000m);  
var a2 = new AutoLicitator(1_000_000m);  
var lic = new Licitatie(10, new IParticipant[] { a1, a2 });  
a1.Licitatie = lic; a2.Licitatie = lic;  
lic.Liciteaza(15);
```

```
Stack overflow.
```

```
Repeat 27507 times:
```

```
    at DesignPatterns.ex9.Models.AutoLicitor.OfertaNoua(System.Decimal)  
    at DesignPatterns.ex9.Models.Licitatie.Liciteaza(System.Decimal)
```

Procesul moare — `StackOverflowException` nu se poate prinde cu `try/catch` în .NET, runtime-ul omoară procesul direct.

DE CE: `Liciteaza` notifică participanții **din interiorul** ei (`Licitatie.cs:31-34`), iar `AutoLicitor.OfertaNoua` cheamă `Liciteaza` înapoi (`AutoLicitor.cs:21`). Fiecare licitație nu se **termină** — se **suspendă**, cu cadrul ei rămas pe stivă, și așteaptă ca licitația declanșată dinăuntru să se termine mai întâi. Cu maxime 100 și 150 lanțul are ~10 pași, deci nu-l vezi. Cu 1.000.000 are 100.000 de pași, iar stiva unui fir .NET e de 1 MB — se umple la ~27.500 de perechi de cadre.

Și e chiar răspunsul empiric la Q3 din runda 4: „cine e vinovat că a ajuns o ofertă învechită?”. Răspunsul e **niciunul dintre ei** — vinovat e faptul că subiectul își cheamă observatorii în timp ce încă e la mijlocul unei modificări. Un Subject corect nu difuzează dintr-o stare intermediară. Tiparul standard: primești oferta, o pui într-o coadă și, dacă nu ești deja în bucla de procesare, o consumi într-un `while` **la nivelul de sus**. Stiva rămâne plată indiferent câți pași are licitația, iar prețurile ies în ordine crescătoare — ceea ce rezolvă și „afișajul minte” (spectatorul vede acum `95, 75, 55, 35, 15` la desfacerea apelurilor).

FIX: vezi tabelul Before/After.

🟡 Importante

M1 — regula pe null s-a oprit exact la cele 4 linii pe care ți le-am citat

`ex7/Models/Canal.cs:7-10` · `ex8/Models/Cont.cs:9-13` · `ex4/Models/StatieMeteo.cs:8-11` ·
`ex5/Models/PoliticaSimpla.cs:9` (+ `PoliticaMedie.cs:9` , `PoliticaPuternica.cs:9`)

M4 din runda 4 enumera patru locuri: `CampParola` ctor, `Vanzare.SchimbaComision` ,
`ComisionCuPlafon` , `ComisionCuBonus` . Toate patru sunt reparate ✅ . **Niciun alt constructor din proiect nu e.** Reprodus:

Apel	Aștept	Primesc
<code>new Canal(null!).PublicaVideo("x")</code>	<code>ArgumentNullException</code> la construcție	<code>NullReferenceException</code> la publicare
<code>new Cont(100, null!).Depune(10)</code>	<code>ArgumentNullException</code> la construcție	<code>NullReferenceException</code> la depunere
<code>new PoliticaSimpla().EsteValida(null!)</code>	<code>ArgumentNullException</code> sau <code>false</code>	<code>NullReferenceException</code>

E al treilea proiect la rând în care apare tiparul ăsta, și e cel mai scump obicei pe care îl ai acum: **repari linia semnalată, nu regula**. Un review îți dă exemple, nu inventar. Când citești „validează la intrare, ca obiectul să nu poată exista într-o stare invalidă”, pasul următor nu e „repar cele 4” — e `Ctrl+Shift+F` după `public Nume(` în toată soluția și o trecere peste fiecare constructor. Îți ia trei minute și închide subiectul definitiv, în loc să-l redeschidem a treia oară.

Nota bună: în `Licitatie.cs:11` ai folosit `ArgumentNullException.ThrowIfNull(participanti)` — forma modernă, o linie, aruncă cu `paramName` corect completat automat. E mai bună decât cele 6 blocuri `if (x == null) throw new ArgumentNullException(nameof(x))` din restul proiectului. Folosește-o peste tot.

Pentru `EsteValida(null)` întrebarea e alta și merită gândită: e `null` o **eroare de programare** (arunci) sau o **parolă invalidă** (întorci `false`)? Ambele răspunsuri se apără; ce nu se apără e `NullReferenceException`, care nu spune nimănui nimic.

M2 — `AutoLicitator.Licitatie` publică și setabilă — a doua rundă neatinsă

`ex9/Models/AutoLicitator.cs:6` + `ex9/Testare9.cs:18-19` + `ex9/Models/Licitatie.cs:16-20`

Compilerul încă îți spune, la fiecare build:

```
warning CS8618: Non-nullable property 'Licitatie' must contain a non-null value when exiting co
```

Și văd în `Licitatie.cs:16-20` că ai încercat, ai pus un `// ???` și ai renunțat:

```

/*for(int i = 0; i < participanti.Length; i++)
{
    var test = participanti[i] as AutoLicitator; // ???
    test.Licitatie = this;
}*/

```

Semnul de întrebare e pus în locul corect, deci hai să-l lămurim. Blocul ăla e greșit din două motive, și **al doilea e cel important**:

1. `as` întoarce `null` când conversia nu merge — pentru `Spectator` (care nu e `AutoLicitator`) `test` iese `null`, iar `test.Licitatie = this` crapă imediat.
2. Mai grav: `Licitatie` ar trebui să știe despre `AutoLicitator`? Nu. Tot Observer-ul stă pe faptul că subiectul cunoaște doar interfața `IParticipant`. În clipa în care faci `as AutoLicitator` ai reintrodus `if`-ul pe tip pe care pattern-ul îl elimină — mâine adaugi `LicitatorUman` și trebuie să te întorci aici.

Soluția care rezolvă și dependența circulară fără să spargă abstracția: **legătura se face la abonare, nu în constructor**. `Licitatie` capătă `Aboneaza(IParticipant p)`, iar `IParticipant` capătă `void SeteazaLicitatie(Licitatie l)` — subiectul se dă pe sine, prin contract; `Spectator` implementează metoda gol, `AutoLicitator` își reține referința. Nimeni nu întreabă „ce fel de participant ești”. Țasta e și motivul pentru care `Aboneaza` / `Dezaboneaza` există în toate celelalte exerciții Observer și lipsesc din ex9.

● Cleanups

- **C1** (C12 rămas) — 11 fișiere fără linie nouă la final: `ex1/Models/Comanda.cs`, `ex1/Models/LivrareCuReducere.cs`, `ex3/Models/ExportMarkdown.cs`, `ex3/Models/Raport.cs`, `ex4/Models/AfisajAlerta.cs`, `ex4/Models/AfisajCurent.cs`, `ex4/Models/AfisajStatistica.cs`, `ex5/Models/CampParola.cs`, `ex6/Models/ComisionPePraguri.cs`, `ex7/Models/Canal.cs`, `ex8/Models/I0bservatorCont.cs`. În Visual Studio: `Tools → Options → Text Editor → C# → Advanced → Automatically add new line at end of file`. Contează la `git diff`: fără ea, orice modificare pe ultima linie apare ca „linie ștearsă + linie adăugată”.
- **C2** — `ex5/Models/PoliticaPuternica.cs:20, :24, :28`: `if (digit == false && Char.IsDigit(...))`. Două lucruri: `digit == false` se scrie `!digit`, iar garda nu-ți economisește nimic real — `Char.IsDigit` e o citire dintr-un tabel. Trei `if` simple, sau `if (Char.IsDigit(p[i])) digit = true;`. Bonus: cu toate trei `true` poți ieși din buclă (`if (digit && bigLetter && notLetterOrDigit) return true;`).

- **C3** — `{ get; private set; }` pe proprietăți care nu se mai schimbă după constructor:
`ComisionFix.cs:5` , `ComisionProcent.cs:5` , `ComisionCuPlafon.cs:7` , `ComisionCuBonus.cs:7` ,
`Vanzare.cs:6` . `{ get; }` spune „imutabil” și îl pune pe compilator să verifice; `private set` spune
„mă schimb, dar doar dinăuntru” — ceea ce nu e adevărat aici. Ai făcut exact mișcarea asta corect la
`Raport.date` (M6). Aceeași regulă.
- **C4** — câmpurile de strategie care nu se schimbă: `ComisionCuPlafon.cs:5` ,
`ComisionCuBonus.cs:5` , `AlertaSoldMic.cs:5` → `readonly` . (`CampParola.politica` și
`Vanzare.strategie` **nu** — alea chiar se schimbă, prin `SchimbaPolitica` / `SchimbaComision` .)
- **C5** — `ex8/Models/Cont.cs:6` și `ex7/Models/Canal.cs:5` : array-urile de observatori se schimbă
doar în `Canal` (prin `Aboneaza`). În `Cont` nu — dar `Cont` nici n-are `Aboneaza` / `Dezaboneaza` , deși
e același pattern ca ex7. Nu e bug; e o asimetrie de care să fii conștient.

Before / After (doar criticele)

B1 — `ex5/Testare5.cs`

Acum	Corect
<pre>Console.Write(politica.Nume + ": "); campParola.Verifica(parola);</pre>	<pre>Console.WriteLine(politica.Nume + ": " + campParola.Verifica(parola));</pre>

Regula: dacă o metodă întoarce ceva, întrebarea „cine citește valoarea asta?” trebuie să aibă răspuns.
Dacă răspunsul e „nimeni”, ori apelul e inutil, ori ai uitat jumătate din fix.

B2 — **ex9** : scoate difuzarea din apelul reintrant

```
public class Licitatie
{
    private readonly IParticipant[] participanti;
    private bool difuzez;           // sunt deja în bucla de notificare?
    private decimal ofertaInAsteptare;
    private bool areOfertaInAsteptare;

    public decimal PretCurent { get; private set; }

    public void Liciteaza(decimal suma)
    {
        if (suma <= PretCurent)
        {
            throw new ArgumentException("The sum is lesser or equal to the current auction bid")
        }

        if (difuzez)                // vin din interiorul unei notificări
        {
            ofertaInAsteptare = suma;    // las oferta și mă întorc – stiva nu crește
            areOfertaInAsteptare = true;
            return;
        }

        difuzez = true;
        PretCurent = suma;

        do
        {
            areOfertaInAsteptare = false;
            for (int i = 0; i < participanti.Length; i++)
            {
                participanti[i].OfertaNoua(PretCurent);
            }

            if (areOfertaInAsteptare)    // cineva a licitat în timpul difuzării
            {
                PretCurent = ofertaInAsteptare;
            }
        }
    }
}
```

```

        while (areOfertaInAsteptare);

        difuzez = false;
    }
}

```

Ce se schimbă: recursivitatea devine **buclă**. Un singur cadru `Liciteaza` pe stivă, oricâți pași ar avea licitația. Și pentru că fiecare rundă de difuzare pornește cu `PretCurent` deja actualizat, nimeni nu mai primește un preț învechit — garda `if (pretCurent != Licitatie.PretCurent) return;` din `AutoLicitator.cs:16` devine redundantă, dar las-o: e o gardă defensivă corectă și nu strică.

Traseul, pas cu pas, ca să vezi diferența față de acum:

	Acum (recursiv)	Cu coadă
<code>Liciteaza(15)</code>	difuzează 15 → auto1 cheamă <code>Liciteaza(25)</code> dinăuntru	difuzează 15 → auto1 lasă 25 în coadă, se întoarce
adâncime stivă la pasul 100	200 de cadre	1 cadru
ce vede Spectatorul	15, 25, ..., 110, apoi 95, 75, 55, 35, 15	15, 25, ..., 110
cu <code>pretMaxim = 1_000_000</code>	<code>Stack overflow</code> , proces omorât	merge

Q&A

Q1. `campParola.Verifica(parola);` a compilat fără nicio eroare și fără niciun warning, deși valoarea întoarsă se pierde. **De ce e asta o decizie deliberată a limbajului și nu o scăpare?** Gândește-te la `list.Remove(x)` — întoarce `bool`, și de cele mai multe ori chiar nu te interesează. Întrebarea care urmează: dacă limbajul nu te poate ajuta aici, **ce anume din felul tău de a lucra ar fi prins-o în 5 secunde?**

Q2. La B2, garda pe preț învechit e corectă, dar tot ai `Stack overflow` la cifre mari. **Formulează în două propoziții diferența dintre „am reparat ce se vede” și „am reparat ce se întâmplă”.** Apoi: mai există în proiect vreun alt loc unde un observator poate chema înapoi în subiect în timpul notificării? (Uită-te la `ex8/Models/Cont.cs:15–21` — ce s-ar întâmpla dacă `AlertaSoldMic` ar face `cont.Depune(...)` ca să acopere descoperitul?)

Q3. M4 din runda 4 enumera 4 locuri fără null-check și le-ai reparat pe toate 4. `Canal` , `Cont` și `StatieMeteo` au aceeași problemă și n-au fost în listă. **Când citești un review, cum decizi dacă o constatare e „linia asta” sau „regula asta”?** Indiciu practic: uită-te la formularea constatării — „`ex5/Models/CampParola.cs:14` nu verifică null” vs „validează la intrare, ca obiectul să nu poată exista într-o stare invalidă”. Care dintre ele e un exemplu și care e o regulă?

Runda 4 — partea A: fix-urile rundei 3 (c8426f8)

B0 (blocant) — proiectul NU COMPILEAZĂ

Build FAILED. 9 Error(s)

Ai redenumit metode în clasele de model, dar **n-ai actualizat apelanții**:

Ai redenumit	Unde a rămas numele vechi
Raport.SchimbaStrategie → SchimbaFormat (ex3/Models/Raport.cs:20)	ex3/Testare3.cs:21 , :25
Raport.Exporta → Genereaza (ex3/Models/Raport.cs:25)	ex3/Testare3.cs:19 , :23 , :27
StatieMeteo.SchimbaStare → SeteazaTemperatura (ex4/Models/StatieMeteo.cs:11)	ex4/Testare4.cs:18 , :20 , :22 , :24

```
error CS1061: 'Raport' does not contain a definition for 'Exporta'
error CS1061: 'StatieMeteo' does not contain a definition for 'SchimbaStare'
```

Redenumirile erau cerute și sunt corecte ca alegere — dar o redenumire **nu e o editare într-un fișier, e o operație peste tot codul care folosește numele**. În Visual Studio: click pe nume → **Ctrl+R, R** (Rename), care schimbă declarația și toate apelurile simultan. Manual n-ai cum să nu ratezi ceva.

Și, mai important: **Ctrl+Shift+B (Build) înainte de fiecare commit**. Nu „am terminat de scris” — „am dat build și trece”. Aici erau 9 erori pe care compilatorul ți le arăta instant, gratuit, înainte de push. E aceeași regulă pe care o are și David acum.

Rezolvate din runda 3

Constatare	Verificat
B1 — <code>ExportCsv</code> duplica primul rând	 <code>ExportCsv.cs:15</code> — bucla pornește de la <code>i = 1</code>
B2 — <code>' , '</code> char în loc de string	 <code>ExportCsv.cs:17</code> — <code>" , "</code>
M1 — <code>AfisajStatistica</code> ținea tot istoricul	 array-ul e scos complet; acum <code>suma</code> + <code>numar</code> , O(1) per actualizare. Și ai scăpat de folosirea lui <code>media</code> ca acumulator
M2 — <code>StatieMeteo</code> cu vocabularul din ex2	 <code>Temperatura</code> / <code>SeteazaTemperatura</code> / <code>afisaje</code>
M3 — strategiile crăpau pe array gol	 <code>if (randuri.Length == 0) return string.Empty;</code> în toate trei
M4 — <code>Testare3</code> ținea tipuri concrete doar pentru <code>.Nume</code>	 și frumos : <code>public string NumeFormat => strategie.Nume; (Raport.cs:7)</code> — expression-bodied property, exact răspunsul corect. Testul nu mai știe ce clase există
C1 (ex1, restanță din 2 runde) — <code>ArgumentOutOfRangeException</code> cu mesajul pe poziția <code>paramName</code>	 <code>LivrareCuReducere.cs:14</code> — <code>nameof(procent)</code> + mesaj
C3 — <code>Raport.Date</code> public	 nu mai e public
C2 — separatorii lipsă din statistică	 parțial, vezi C7

Nouă din nouă pe fond. Toate fix-urile sunt **corecte** — problema e exclusiv că n-ai dat build după.

Importante (partea A)

M6 — `Raport.Date` e o proprietate privată, nu un câmp

`ex3/Models/Raport.cs:6`

```
private string[] Date { get; }
```

Ai rezolvat C3 mutând-o pe `private`, dar o **proprietate** privată auto-implementată e un câmp cu ceremonie în plus: în spate compilatorul tot generează un câmp, plus un getter pe care nu-l vede nimeni. Dacă nu e vizibilă din afară, e câmp:

```
private readonly string[] date;
```

Convenția C#: proprietăți `PascalCase` pentru ce e public, câmpuri `camelCase` pentru ce e intern. `readonly` spune explicit ce voiai să spui cu `{ get; }` — se setează în constructor și nu se mai schimbă.

M7 — **Raport** validează **date**, dar nu și **strategie**

`ex3/Models/Raport.cs:11-17`

Aceeași jumătate-de-gardă ca la M4 din partea B: dacă `strategie` vine `null`, obiectul se construiește fără să se plângă și crapă abia la `Genereaza()`, departe de locul greșelii. Și `SchimbaFormat ( :20 )` nu verifică nimic.

● Cleanups (partea A)

- **C7** — `ex4/Models/AfisajStatistica.cs:24` : `"[STATISTICA] Min: " + minim + " | Max: "` — bara e lipită de valoarea din stânga: `Min: 20 | Max: 40`. Vrei `" | Max: "`.
- **C8** — `ex4/Models/AfisajStatistica.cs:6` : `private decimal numar = 0;` — e un **contor de măsurători**, nu o valoare zecimală. `int numar` . (`suma / numar` merge oricum: `decimal / int` se promovează la `decimal` .)
- **C9** — `ex3/Models/ExportCsv.cs:1` : ai adăugat `using System.Text;` dar nu folosești `StringBuilder` nicăieri — ai început C4 din runda 3 și l-ai abandonat. Ori duci fix-ul până la capăt, ori scoți `using` -ul.
- **C10** — `ex4/Models/StatieMeteo.cs:8` : ai redenumit câmpul în `afisaje`, dar parametrul constructorului a rămas `observatori`. Redenumirea s-a oprit la jumătatea liniei.
- **C11** — `ex3/Models/Raport.cs:13` : `ArgumentNullException("date")` e **corect** (aici primul argument chiar e `paramName`), dar acum că ai învățat `nameof` la C1 — `nameof(date)`. Avantajul: dacă redenumesc parametrul, se redenumesc și string-ul. Exact problema de la B0.
- **C12** — `ex4/Models/AfisajStatistica.cs` : fișierul nu se termină cu linie nouă (`\ No newline at end of file` în diff).

Runda 4 — partea B: caietul de exerciții S1, S2, S3, O1, O2, O3 (ex5–ex9)

Constatările de mai jos sunt verificate prin rulare pe commit-ul `f07b7f3`, adică **înainte** ca redenumirile din `c8426f8` să spargă build-ul (B0). Sunt valabile ca atare — ex5–ex9 n-au fost atinse de acel commit.

Cinci exerciții din caiet, toate solo, fără pași dați. Structura e bună peste tot: contract → implementări → context. Dar de data asta **codul nu e doar „de îmbunătățit” — două exerciții din cinci sunt rupte funcțional**, iar unul dintre ele crapă la prima rulare.

Prima observație, înainte de orice diagnostic: **Program.cs:15–23 are toate testările comentate în afară de Testare9**. Asta înseamnă că `Testare5` ... `Testare8` n-au mai fost rulate după ce le-ai scris. Iar `Testare7` nici măcar nu apelează `Aboneaza`. Regula rămâne cea de la David: *nu comiți ce n-ai rulat*, iar la un caiet de exerciții „am rulat” înseamnă **fiecare** exercițiu, nu ultimul.

Ce e corect ✓

- **S3 (ex6) — Decorator compus, calculat corect de la interior spre exterior.**
`ComisionCuPlafon(200, ComisionProcent(20))` pe 1500 → `min(300, 200) = 200`, apoi `ComisionCuBonus(50, plafon)` → 250. Exact cerința. Și ai prins ideea că decoratorul implementează **același** contract pe care îl consumă — de aia se pot înlănțui.
- **ComisionPePraguri** **dă 100 la 1500** ($5\% \cdot 1000 + 10\% \cdot 500$) — verificat manual pe formula din `ex6/Models/ComisionPePraguri.cs:9–15`.
- **CampParola** și **Vanzare** **n-au niciun if pe „ce fel de strategie”**. Miezul Strategy e prins curat în ambele.
- **ex8 — validările și demonstrarea lor.** `ArgumentException` pe sumă negativă, `InvalidOperationException` pe fonduri insuficiente, iar `Testare8.cs:21–28` chiar declanșează cazul de eroare cu `try/catch`. Asta era M2 din runda 1 și M2 din runda 2 — acum o faci din reflex, fără să ți-o ceară nimeni.
- **ex9 — ai intuit singur riscul de auto-supralicitare** și ai pus o gardă (`ultimPret != pretCurent`, `AutoLicitator.cs:16`). E incompletă (vezi B3), dar întrebarea ți-ai pus-o corect.
- **Collection expressions** `[ ... ]` (`Testare7.cs:10–14`, `Testare8.cs:9–13`, `Testare9.cs:11–15`) — sintaxă C# 12, idiomatică.

B1 — Aboneaza iese din array: **IndexOutOfRangeException** garantat`ex7/Models/Canal.cs:14-22`

```
IAbonat[] tempAbonati = new IAbonat[abonati.Length + 1];

for (int i = 0; i < tempAbonati.Length; i++) // ← tempAbonati, nu abonati
{
    tempAbonati[i] = abonati[i];           // ← la ultima iterație: abonati[abonati.Length]
}
```

Copiezi din `abonati` dar te oprești după lungimea lui `tempAbonati`, care e cu 1 mai mare. Ultima iterație citește `abonati[abonati.Length]` — o poziție care nu există.

Reprodus izolat: `Canal` cu 2 abonați + `Aboneaza(al treilea)` → `IndexOutOfRangeException`.

Metoda **nu e apelată niciodată în Testare7** — de asta n-ai văzut-o. Bonusul scris dar nedemonstrat: aceeași constatare ca M2 runda 1, la ex2. Diferența e că atunci codul era corect și doar nedemonstrat; acum nedemonstrarea ascunde un crash.

B2 — Dezaboneaza alocă array-ul după numărul **GREȘIT** — pierzi abonați`ex7/Models/Canal.cs:25-45`

```
int cnt = 0;
for(int i = 0; i < abonati.Length; i++)
{
    if(abonati[i] == vechi) cnt++; // ← numeri câți SE SCOT
}

IAbonat[] tempAbonati = new IAbonat[cnt]; // ← dar aloci pentru câți RĂMÂN
cnt = 0;
for(int i = 0; i < tempAbonati.Length; i++) // ← și parcurgi doar cnt poziții din abonati
{
    if (abonati[i] != vechi) tempAbonati[cnt++] = abonati[i];
}
```

Trei greșeli care se compun: numeri potrivirile în loc de rămășițe, dimensionezi după ele, apoi parcurgi sursa după lungimea destinației.

Reprodus izolat:

Situația	Ce ar trebui	Ce se întâmplă
3 abonați <code>[a, b, c]</code> , <code>Dezaboneaza(b)</code>	rămân <code>a</code> , <code>c</code>	rămâne doar <code>a</code> — <code>c</code> dispare
3 abonați, <code>Dezaboneaza(cineva</code> <code>neabonat)</code>	rămân toți 3	array gol — pierzi toți abonații
2 abonați <code>[email, push]</code> , <code>Dezaboneaza(push)</code>	rămâne <code>email</code>	rămâne <code>email</code> 

Ultimul rând e motivul pentru care `Testare7` trece: cu exact 2 abonați și exact 1 scos, `cnt` nimerește din întâmplare valoarea corectă (1). Un singur caz de test, ales fix pe coincidența care ascunde bug-ul.

Și acum partea care contează: **asta e B1 din runda 1 (ex2), pe care ai reparat-o corect atunci**. Review-ul rundei 3 confirma: „acum numeri întâi, apoi aloci — exact traseul corect”. Ai știut traseul, dar l-ai reconstruit din memorie în loc să te uiți la ce scrisesei, și ai numărat celălalt lucru. Când reîntâlnești un pattern pe care l-ai rezolvat deja, **deschide soluția veche** — nu e trișat, e exact ce faci în producție.

B3 — `ex9` crapă: bucla de feedback difuzează prețuri învechite

`ex9/Models/Licitatie.cs:25-28` + `ex9/Models/AutoLicitator.cs:14-29`

`dotnet run` pe `Testare9` , output real:

```
[SPECTATOR] Pret curent: 25
[SPECTATOR] Pret curent: 45
[SPECTATOR] Pret curent: 65
[SPECTATOR] Pret curent: 85
[SPECTATOR] Pret curent: 100
[SPECTATOR] Pret curent: 110
[SPECTATOR] Pret curent: 95          ← prețul SCADE
Unhandled exception. System.ArgumentException: The sum is lesser or equal to the current action
    at Licitatie.Liciteaza(Decimal suma) in Licitatie.cs:line 21
```

Mecanismul: `AutoLicitator.OfertaNoua` cheamă `Licitatie.Liciteaza` **din interiorul buclei `for` care încă notifică**. Apelul imbricat schimbă `PretCurent` (110) și se întoarce — dar bucla exterioară continuă să difuzeze `suma` cu care pornise ea (95). Al doilea licitator primește 95, crede că licitația e la 95, licitează 105 — și `Liciteaza` respinge, pentru că prețul real e deja 110. Excepția nu e prinsă nicăieri → programul moare.

Caietul te avertiza la O3: „*ce oprește lanțul?*”. `pretMaxim` oprește **escaladarea**, dar nu rezolvă a doua problemă: cât timp e în curs o notificare, prețul pe care îl primești poate fi deja vechi.

B4 — `<=` în loc de `<` la `pretMaxim` — a doua cale de crash

`ex9/Models/AutoLicitator.cs:16`

```
if(pretCurent <= pretMaxim && ultimPret != pretCurent)
```

Cerința spune `pretCurent < pretMaxim`. Cu `<=`, când prețul ajunge **exact** la `pretMaxim`, licitatorul intră în `if`, calculează `pretCurent + 10 > pretMaxim` → true, și licitează `pretMaxim` — adică **exact suma curentă**. `Liciteaza` aruncă `ArgumentException`.

În rularea de acum e mascat: `auto1` ajunge singur la 100, deci `ultimPret == 100` îl oprește. Dar dacă `auto2` ar fi cel care licitează 100, `auto1` primește 100 cu `ultimPret` diferit → intră → crapă. Bug latent, salvat doar de ordinea din test.

🟡 Importante

M1 — `CampParola.Verifica` întoarce `void`, deși contractul cerea `bool`

`ex5/Models/CampParola.cs:21-24`

```
public void Verifica(string parola)
{
    Console.WriteLine(politica.EsteValida(parola));
}
```

Caietul: „`Verifica(string parola)` **întoarce** `true/false`”. Contextul unui Strategy livrează rezultatul apelantului; nu decide el că rezultatul se afișează pe consolă. Așa cum e acum nu poți folosi `CampParola` într-un formular, într-un API sau într-un test — singurul lucru pe care îl știe face e să scrie pe ecran.

M2 — politicile din ex5 afișează mesaje din interiorul validării

ex5/Models/PoliticaSimpla.cs:11 · PoliticaMedie.cs:11,23 ·
PoliticaPuternica.cs:11,38,43,48

Aceeași problemă, un nivel mai jos: strategia decide **și** comunică. Consecință concretă în rularea ta — output-ul iese amestecat, pentru că `Console.Write(ume + ": ")` din test e urmat de mesajul de eroare al politicii și abia apoi de `True/False` :

```
Medie: The password must contain at least 8 characters  
False
```

Forma corectă: `EsteValida` întoarce doar verdictul (eventual și un motiv, printr-un `out string motiv` sau un mic `RezultatValidare`), iar cine a cerut validarea decide dacă îl scrie pe consolă, îl pune sub input în UI sau îl loghează. Regula generală: **o clasă care calculează nu face I/O.**

M3 — bucla de notificare duplicată în `Cont`

ex8/Models/Cont.cs:22–25 și ex8/Models/Cont.cs:40–43

Aceleași 4 linii, copiate în `Depune` și `Retrage`. La a treia operațiune (transfer, dobândă) o copiezi a treia oară și una din ele va rămâne în urmă. Extrage `private void NotificaObservatori()` și cheam-o din ambele — e chiar metoda pe care Observer o presupune în Subject.

M4 — null-check inconsistent de la un exercițiu la altul

- ex5/Models/CampParola.cs:14 — `SchimbaPolitica` verifică `null` , dar **constructorul** (linia 7) nu.
- ex6/Models/Vanzare.cs:14–17 — `SchimbaComision` nu verifică deloc.
- ex6/Models/ComisionCuPlafon.cs:9 și `ComisionCuBonus.cs:9` — validează plafonul, dar nu și strategia împachetată; un `null` acolo crapă abia la `Calculeaza`, departe de locul greșelii.

Principiul: validează la **intrare** (constructor și setter deopotrivă), ca obiectul să nu poată exista într-o stare invalidă. Jumătate de gardă e mai rea decât niciuna, pentru că îți dă impresia că e acoperit.

M5 — `AutoLicitator.Licitatie` e o proprietate publică setată după construcție

ex9/Models/AutoLicitator.cs:6 + Testare9.cs:18–19

Compilerul îți spune deja:

```
warning CS8618: Non-nullable property 'Licitatie' must contain a non-null value when exiting co
```

Între `new AutoLicitator(100)` și `auto1.Licitatie = licitatie` obiectul e într-o stare în care orice `OfertaNoua` dă `NullReferenceException`. Aici e o dependență circulară reală (licitația are nevoie de participanți, participanții de licitație) — soluția uzuală e ca `Licitatie` să se dea pe sine la abonare (`licitatie.Aboneaza(auto1)` setează legătura), nu să lase apelantul să-și amintească două linii separate.

Warning-urile de build se citesc. Țsta indica exact spre problemă.

Cleanups

- **C1** — `ex9/Models/Licitatie.cs:1` : `using System.Security.Cryptography.X509Certificates;` nefolosit (autocomplete accidental pe „X”). Șterge-l.
- **C2** — `ex5/Models/PoliticaPuternica.cs:22-33` : lanțul `else if`. Funcționează, dar **din noroc**: un caracter nu poate fi simultan cifră, majusculă și special, deci ramurile nu se fură între ele. Trei `if` independente exprimă intenția („verific fiecare cerință separat”) și nu depind de coincidența asta. Plus liniile 18-19, goale dar cu spații.
- **C3** — `ex6/Models/ComisionCuBonus.cs:7` : proprietatea se numește `Comision`, dar conține **bonusul**. Într-o clasă care implementează `IComision` și are `Calculeaza`, numele ăsta induce în eroare. `Bonus`.
- **C4** — `ex8/Models/AlertaSoldMic.cs:14` : ai `soldNou <= prag`, tabelul din caiet spune `<`. **Aici caietul se contrazice pe el însuși** (testul cere ca la sold 50 cu prag 50 alerta să pornească, ceea ce cere `<=`). Nu e greșeala ta — o lămurim la lecție.
- **C5** — `ex6/Models/ComisionFix.cs:8` : fără validare pe sumă negativă, deși `ComisionProcent` și `ComisionCuPlafon` validează. Vezi M4.
- **C6** — `ex5/Models/CampParola.cs:16` : `throw new ArgumentNullException("new policy")` — primul argument al lui `ArgumentNullException` e `paramName`, deci mesajul afișat va fi „Parameter 'new policy'”. Corect: `nameof(noua)`. **E a treia rundă în care apare aceeași subtilitate** (C1 ex1, apoi nota din runda 3). Regula, încă o dată: `ArgumentException(mesaj, paramName)` — mesajul primul; `ArgumentNullException(paramName, mesaj)` și `ArgumentOutOfRangeException(paramName, mesaj)` — numele primul.

Before / After (doar criticele)

B1 — Canal.Aboneaza

Acum	Corect
<pre>for (int i = 0; i < tempAbonati.Length; i++) tempAbonati[i] = abonati[i];</pre>	<pre>for (int i = 0; i < abonati.Length; i++) tempAbonati[i] = abonati[i];</pre>

Regula: **bucla de copiere merge după lungimea SURSEI, nu a destinației.**

B2 — Canal.Dezaboneaza

```
public void Dezaboneaza(IAbonat vechi)
{
    int ramasi = 0;
    for (int i = 0; i < abonati.Length; i++)
    {
        if (abonati[i] != vechi) ramasi++;    // numeri cine RĂMÂNE
    }

    IAbonat[] tempAbonati = new IAbonat[ramasi];
    int poz = 0;
    for (int i = 0; i < abonati.Length; i++)    // parcurgi SURSA integral
    {
        if (abonati[i] != vechi) tempAbonati[poz++] = abonati[i];
    }

    abonati = tempAbonati;
}
```

Doi indici cu roluri diferite (`i` citește, `poz` scrie) — de-asta refolosirea lui `cnt` te-a încurcat.

B3 + B4 — AutoLicitator.OfertaNoua

Acum	Corect
<code>if(pretCurent <= pretMaxim && ultimPret != pretCurent)</code>	vezi mai jos

```
public void OfertaNoua(decimal pretCurent)
{
    if (pretCurent != Licitatie.PretCurent) return; // B3: ofertă învechită, ignor-o
    if (ultimPret == pretCurent) return;           // nu mă supralicitez pe mine
    if (pretCurent >= pretMaxim) return;           // B4: strict mai mic

    ultimPret = Math.Min(pretCurent + 10, pretMaxim);
    Licitatie.Liciteaza(ultimPret);
}
```

Prima linie e miezul: dacă prețul pe care l-ai primit nu mai e prețul real al licitației, notificarea e depășită — taci. Cele trei `return` timpurii înlocuiesc și `if`-ul imbricat din `AutoLicitator.cs:18-27`.

Verificat prin rulare: licitația urcă 25 → 45 → 65 → 85 → 100 → 110 și se oprește singură la 110, **fără excepție**.

Un rest de discutat (nu e bug, e Q3): spectatorul vede după aceeași `95, 75, 55, 35, 15` — prețurile vechi difuzate pe măsură ce apelurile imbricate se desfac. Prețul e corect, afișajul minte.

Q&A

Q1. `Dezaboneaza` din `ex7` are exact bug-ul pe care l-ai reparat în `ex2` acum două runde — dar pe dos: acolo presupuneai că scoți exact unul, aici numeri câți scoți în loc de câți rămân. Testul tău cu 2 abonați trece în ambele variante. **Ce test cu un singur apel în plus ar fi prins ambele bug-uri deodată?** (Indiciu: câți abonați îți trebuie minim, și ce se întâmplă dacă dezabonezi pe cineva care nu e abonat?)

Q2. În `ex5`, `PoliticaMedie.EsteValida` face două lucruri: decide dacă parola e validă și scrie pe consolă de ce nu e. Presupune că mâine aceeași politică trebuie folosită într-un API REST, care întoarce JSON. **Ce trebuie să întorci din `EsteValida` ca să meargă în ambele locuri, fără să duplici regulile?**

Q3. Caietul întreba la Q3: *ce risc apare când observatorii pot modifica sursa pe care o ascultă?* Acum ai și răspunsul empiric — programul a crăpat. **De ce nu e suficient ca `Licitatie` să verifice `suma <= PretCurent` și să arunce excepție?** Altfel spus: cine e vinovat că a ajuns o ofertă învechită — cel care o

trimite sau cel care o primește? Și ce ar trebui să facă `Liciteaza` cu notificările cât timp o notificare e deja în curs?

Q4. (partea A) Ai redenumit `Exporta` → `Genereaza` și build-ul a picat cu 9 erori. Numele vechi mai era într-un singur fișier, pe care îl aveai deschis. **Ce te-ar fi oprit să comiți asta — și de ce `Ctrl+R, R` din Visual Studio nu e doar „mai comod”, ci mai sigur decât să cauți manual numele?** Bonus, legat de C11: dacă în loc de `nameof(date)` scrii `"date"` și apoi redenumеști parametrul, cine te avertizează?

Runda 3 — ex3 (Strategy solo) + ex4 (Observer solo)

Rezolvate din runda 2

Constatare	Verificat
B1 — <code>Dezaboneaza</code> dimensiona array-ul din presupuneri	 acum numeri întâi (<code>Comanda.cs:29-38</code>), apoi aloci — exact traseul corect
B2 — formula <code>LivrareStandard</code> avea coeficienții Express	 <code>5 + 0.5m * g + 0.1m * d</code>
M2 (ex1) — cazul de eroare nedemonstrat	 <code>Testare1.cs:29-37</code> , <code>try/catch</code> (<code>ArgumentException</code>) cu greutate <code>0</code>
M2 (ex2) — bonusul <code>Aboneaza</code> / <code>Dezaboneaza</code> nedemonstrat	 <code>Testare2.cs</code> abonează după prima schimbare și dezabonează după a doua
M1 — <code>.vs/</code> în git	 scos din tracking + <code>.gitignore</code>
C1 — spațiere <code>i <</code>	
Notă de design — notificare fără schimbare reală	 <code>Comanda.cs:53</code> — <code>if (Stare == stareNoua) return;</code>

Șase din șase, plus nota opțională. Asta e prima rundă în care n-ai introdus nicio regresie reparând altceva — exact lecția din Q2 runda 2.

Rămas nerezolvat: C1 din ex1 — `ArgumentOutOfRangeException("Discount percent...")` (`LivrareCuReducere.cs:14`) tot trimite mesajul pe poziția lui `paramName` . Rulează-l și uită-te la textul afișat: „Parameter 'Discount percent must be between 0 and 100'”. Corect: `new` `ArgumentOutOfRangeException(nameof(procent), "Discount percent must be between 0 and 100")` .

Ce e corect în runda asta

- **Ambele pattern-uri aplicate solo, fără pași dați.** `Raport` nu are niciun `if` pe format; `StatieMeteo` nu cunoaște nicio clasă concretă de afișaj. Miezul e prins în amândouă.
- **AfișajStatistica cu stare proprie** care chiar se acumulează — media se mișcă 20 → 25 → 30 la rulare. Observatori cu stare, nu doar printere.
- `minim = decimal.MaxValue / maxim = decimal.MinValue` ca valori de start — idiomul corect, l-ai găsit singur (alternativa naivă, `minim = 0`, ar fi dat minim 0 la temperaturi pozitive).
- `ArgumentNullException("date")` (`Raport.cs:12`) — aici primul argument CHIAR e `paramName`, deci e corect. Fix subtilitatea de la C1: `ArgumentException` primește mesajul primul, `ArgumentNullException` și `ArgumentOutOfRangeException` primesc numele parametrului.
- **N-ai portat garda** `if (Stare == stareNoua) return;` în `StatieMeteo`. Bine — vezi Q1, nu e o omisiune, e o decizie corectă.

Critice

B1 — `ExportCsv` duplică primul rând și pune virgulă în față

`ex3/Models/ExportCsv.cs:11-16`

Adaugi `randuri[0]` înainte de buclă, dar bucla pornește tot de la `0`:

```
rez += randuri[0];

for (int i = 0; i < randuri.Length; i++)
{
    rez += ',' + randuri[i];
}
```

La rulare iese:

```
CSV:
bia,bia,este,o prietena,foarte buna
```

„bia” apare de două ori. La `ExportText:13` și `ExportMarkdown:13` ai scris exact aceeași formă — element zero în afara buclei, buclă de la `1` — și acolo e corect. Doar CSV a rămas cu `i = 0`, probabil pentru că la CSV separatorul e la fel peste tot și nu s-a „simțit” diferența la scris.

	Acum	Corect
Cod	<pre>rez += randuri[0]; for (int i = 0; ...) rez += ',' + randuri[i];</pre>	<pre>rez += randuri[0]; for (int i = 1; ...) rez += "," + randuri[i];</pre>
Output	<pre>bia,bia,este,o prietena, foarte buna</pre>	<pre>bia,este,o prietena, foarte buna</pre>

Lecția: tehnica „primul element în afara buclei” are întotdeauna două jumătăți — scoți elementul 0 și pornești bucla de la 1. Dacă schimbi doar una, se dublează tăcut: nu e nicio eroare de compilare, nicio excepție, doar un rezultat greșit. Când repeți același șablon în trei clase, verifică-le una lângă alta — divergențele de o cifră se văd doar în comparație. (Enunțul îți permitea `string.Join(",", randuri)` la CSV — o singură linie, fără index de greșit.)

B2 — , ca char face adunare de numere, nu concatenare

ex3/Models/ExportCsv.cs:15

Ascuns în B1, dar e o capcană separată, care te va prinde din nou:

```
rez += ',' + randuri[i];
```

`','` e `char`, nu `string`. C# evaluează întâi `',' + randuri[i]` — și fiindcă `char` e un tip numeric, într-un context unde operandul e `string` se face totuși concatenare, deci aici scapi. Dar scrie `rez += ',' + '-'`; și nu mai obții `","`, obții `89` (44 + 45, codurile numerice adunate). Regula sigură: separatorii se scriu `","`, cu ghilimele duble, nu `','`.

Lecția: `char` e un număr în C#. `+` între două `char`-uri e adunare aritmetică, nu concatenare — devine concatenare doar dacă cel puțin un operand e `string`. Aici te-a salvat `randuri[i]`.

🟡 Importante

M1 — AfisajStatistica ține tot istoricul, deși cerința îți spunea să ții doar câmpuri

ex4/Models/AfisajStatistica.cs:5,12,25–33

Păstrezi un `decimal[] temperaturi` pe care îl recreezi la fiecare măsurătoare, doar ca să recalculezi suma de la zero:

```

media = 0;
for (int i = 0; i < temperaturi.Length; i++)
{
    tempNoi[i] = temperaturi[i];
    media += temperaturi[i];
}

```

Rezultatul e corect, dar constrângerea din enunț era explicită: „*pentru statistica tine tu campurile (min, max, suma, numar)*”. Cu `suma` și `numar` ca stare, `Actualizeaza` devine trei linii, fără array, fără buclă: `suma += temperatura; numar++; media = suma / numar;`. Diferența practică: acum, la măsurătoarea N faci N copieri și N adunări — după 10.000 de măsurători, o stație meteo reală ar face 50 de milioane de operații pentru o medie care cere una singură.

Al doilea lucru, mai important decât viteza: folosești câmpul `media` pe post de acumulator temporar (`media = 0;` apoi `media += ...`). Câtă vreme bucla merge, `media` conține o sumă parțială, nu o medie — un câmp care minte despre ce ține între două linii de cod. Un acumulator care trăiește doar în interiorul metodei trebuie să fie variabilă locală; câmpurile țin doar starea care are sens și după ce metoda s-a terminat.

Lecția: min/max/media sunt agregate *incrementale* — se pot actualiza din valoarea nouă plus starea de dinainte, fără să reții istoricul. Ai văzut deja asta la min și max (le actualizezi corect, cu un `if`); media urmează exact aceeași logică, dacă ții `suma` și `numar` în loc de valori. Reține istoricul doar când chiar ai nevoie de el — de exemplu la un `AfisajGrafic` care desenează evoluția.

M2 — `StatieMeteo` a rămas cu numele din ex2

`ex4/Models/StatieMeteo.cs:5,6,13`

Ai copiat scheletul din `Comanda` și n-ai schimbat vocabularul: `observatori`, `Stare`, `SchimbaStare`. Enunțul cerea `SeteazaTemperatura(decimal t)`, iar domeniul vorbește despre afișaje și temperatură, nu despre stări și observatori.

```

private IAfisaj[] observatori;
public decimal Stare { get; private set; }
public void SchimbaStare(decimal stareNoua)

```

Contează pentru că `Stare` e acum un `decimal` care se cheamă „stare” — cine citește `statie.SchimbaStare(30)` nu are cum să ghicească dacă 30 e grade, minute sau un cod. Numele corecte (`afisaje`, `Temperatura`, `SeteazaTemperatura`) fac codul auto-explicativ fără niciun comentariu.

Lecția: pattern-ul se refolosește, vocabularul nu. Structura lui Observer e aceeași în orice domeniu — un subiect, o listă de abonați, o buclă de notificare — dar numele trebuie să vină din domeniul curent. Când numele rămân din exercițiul precedent, e semnul că ai copiat forma fără să retraduci problema.

M3 — cele 3 strategii crapă pe un array gol

`ex3/Models/ExportCsv.cs:11` , `ExportText.cs:11` , `ExportMarkdown.cs:11`

Toate trei citesc `randuri[0]` înainte de orice verificare. `new Raport(new string[0], new ExportCsv())` trece de validarea din constructor (nu e `null`) și crapă la `Exporta()` cu `IndexOutOfRangeException`. Ai apărat repotul de `null`, dar nu de „gol” — și gol e cazul mult mai probabil în practică (un raport fără rânduri e o situație normală, nu o eroare de programare). Un `if (randuri.Length == 0) return string.Empty;` la începutul fiecărui `Formateaza` închide subiectul.

Lecția: `null` și „gol” sunt două lucruri diferite, și se apără diferit. `null` înseamnă „lipsește obiectul” — de obicei un bug al apelantului, deci excepție. Gol înseamnă „obiectul există, dar n-are elemente” — de obicei un caz de business valid, deci un rezultat gol, nu o excepție.

M4 — **Testare3** ține referințe la strategiile concrete doar ca să afișeze **Nume**

`ex3/Testare3.cs:17,22,27`

```
ExportText text = new();
raport.SchimbaStrategie(text);
Console.WriteLine(text.Nume + ":\n" + raport.Exporta());
```

Declari variabile de tip concret (`ExportCsv`, `ExportText`, `ExportMarkdown`) și le ții în viață după ce le-ai dat raportului, pentru un singur motiv: să citești `.Nume`. Dar cine știe formatul curent e `Raport`-ul — el ține strategia. Cu o proprietate `public string NumeFormat => strategie.Nume;` în `Raport`, testul devine `raport.SchimbaStrategie(new ExportText());` urmat de `raport.NumeFormat` — și nu mai are nevoie de tipuri concrete deloc.

Lecția: dacă apelantul trebuie să țină minte ce i-a dat contextului ca să poată descrie starea contextului, informația stă în locul greșit. Un context complet își poate răspunde singur la „cu ce lucrezi acum?”. Verificarea rapidă: dacă poți schimba toate variabilele din test din tip concret în tipul interfeței și codul tot compilează, contextul e complet — acum `ExportCsv strategie` nu poate deveni `IExportStrategie strategie` fără să pierzi nimic, ceea ce e semnul că `.Nume` e citit de unde nu trebuie.

Cleanups

- **C1** — `Raport.SchimbaStrategie / Exporta` vs. `SchimbaFormat / Genereaza` din enunț (`Raport.cs:19,24`). Codul e corect; dar când enunțul fixează un nume, respectă-l — la lucrul în

echipă contractul e literă de lege, nu sugestie.

- **C2** — formatul cerut la statistică era `[STATISTICA] Min: a | Max: b | Media: c`; tu afișezi fără separatorii `|` (`AfisajStatistica.cs:35`). Cosmetic, dar output-ul cerut face parte din cerință.
- **C3** — `Raport.Date` e `public` și întoarce chiar array-ul intern (`Raport.cs:6`); oricine îl primește poate schimba conținutul raportului pe la spate. Aici nimeni n-o face, dar câmpul n-are motiv să fie `public`.
- **C4** — concatenare cu `+=` în buclă (toate cele 3 strategii): fiecare `+=` creează un string nou, fiindcă în C# string-urile sunt imutabile. La 4 rânduri e irelevant; la 10.000 se simte. `StringBuilder` e unealta pentru asta — nu e interzisă de enunț, spre deosebire de LINQ și `List`.
- **C5** — `StatieMeteo` nu verifică `observatori == null` (`StatieMeteo.cs:10`), deși în `Raport` ai pus verificarea. Aceeași grijă, aplicată inconsecvent între cele două exerciții.

Q&A

Q1. În `Comanda` (ex2) ai adăugat `if (Stare == stareNoua) return;` — corect acolo. În `StatieMeteo` nu l-ai pus. Presupune că îl pui: ce se întâmplă cu media din `AfisajStatistica` dacă stația măsoară 30, apoi tot 30? De ce garda e corectă la o comandă și greșită la o stație meteo?

Q2. `ExportText` și `ExportMarkdown` funcționează, `ExportCsv` nu — deși toate trei sunt scrise după aceeași schemă. Uită-te la ele una sub alta și spune ce diferență de un singur caracter face separarea. Apoi: ce ai fi putut face la scris ca greșeala să nu fie posibilă deloc? (hint: ce anume trebuie repetat de trei ori, și ce nu)

Q3. `AfisajCurent` n-are niciun câmp; `AfisajStatistica` are patru. Amândouă intră în același `IAfisaj[]`. Dacă mâine adaugi `AfisajGrafic` care chiar are nevoie de tot istoricul, câte linii din `StatieMeteo` atingi — și de ce răspunsul ăsta e tot argumentul pentru care Observer merită efortul?

ex2 — Observer (runda 1)

Ce e corect ✓

- **Observer aplicat corect.** `SchimbaStare` setează `Stare` și parcurge observatorii cu `for`, chemând `Actualizeaza` — fără să cunoască nicio clasă concretă. Exact miezul pattern-ului.
- **Toate cerințele 1-3 îndeplinite** — și output-ul respectă formatul din enunț (`[EMAIL] ...`, `[LOG] ...`, `[DEPOZIT] ...`), grupat câte 3 pe schimbare. Progres clar față de `Testare1` din prima rundă, care era incomplet.
- **Bonus Aboneaza** — crești array-ul manual, corect, fără `List`.

● Critice

B1 — `Dezaboneaza` presupune că observatorul există EXACT o dată

`ex2/Models/Comanda.cs:29`

Pre-dimensionezi array-ul nou la `observatori.Length - 1` ÎNAINTE să știi câte apariții scoți:

```
IObserver[] obsNou = new IObserver[observatori.Length - 1];
```

Dacă `vechi` **nu e în listă**, toate cele N elemente trec de `if (observatori[i] != vechi)` și încerci să pui N elemente într-un array de N-1 → `IndexOutOfRangeException` la ultima copiere. Dacă `vechi` apare **de două ori**, rămâne un slot `null` la coadă, iar la următoarea `SchimbaStare` crapă cu `NullReferenceException`. Și fiindcă nu chemi `Dezaboneaza` nicăieri (vezi M2), bug-ul e ascuns — netestat.

	Acum	Corect
Idee	dimensionezi întâi, presupui o apariție	numeri întâi câte rămân, apoi dimensionezi

Un traseu sigur: parcurgi o dată și NUMERI câte sunt `!= vechi`, aloci array-ul de acea mărime, apoi copiezi. Așa merge și când lipsește, și când apare de mai multe ori.

Lecția: când mărimea rezultatului depinde de conținut, n-o fixa din presupuneri — calculează-o din date. „Sigur e exact unul” e felul în care se nasc `IndexOutOfRangeException`-urile.

● Importante

M1 — `.vs/` a ajuns în git (15 fișiere)

Repo. `git add .` a măturat folderul ascuns al Visual Studio (`.vs/` — setări locale de IDE, cache, layout de ferestre). Nu e cod, e starea editorului TĂU; pe altă mașină n-are sens. `.gitignore` are doar `bin/` și `obj/`. Aduagă `.vs/` și scoate-l din tracking:

```
git rm -r --cached .vs
```

De acum, regula fixă: orice folder pe care NU l-ai scris tu (îl generează IDE-ul sau compilatorul) intră în `.gitignore` ÎNAINTE de primul `git add .`

M2 — bonusul e scris dar nedemonstrat în `Testare2`

`ex2/Testare2.cs`

`Aboneaza` / `Dezaboneaza` există, dar `Testare2` nu le cheamă niciodată. Cerința bonus cerea explicit: abonează un observator DUPĂ prima `SchimbaStare` și arată că prinde doar schimbările următoare. Fără scenariul ăsta, bonusul e cod nedovedit — și exact asta a ținut ascuns bug-ul B1. Aduagă: după `SchimbaStare("Expediata")`, un `Aboneaza(...)`, apoi `SchimbaStare("Livrata")`, și un `Dezaboneaza(...)` de probă.

● Cleanups

- **C1** — spațiere dublă `i < observatori.Length` (`Comanda.cs:45`). Cosmetic.
 - **Notă de design** — `SchimbaStare` anunță chiar dacă starea nouă e identică cu cea veche. Enunțul menționa capcana „notificare fără schimbare reală”. Opțional: dacă `stareNoua == Stare`, nu mai anunța.
-

ex1 — Strategy (runda 2)

Rezolvate

Din runda 1	Verificat
M2 (parțial) — <code>Nume</code> în output + <code>LivrareGratuita</code> testată	 toate 4 strategiile apar etichetate
M3 — excepție corectă în <code>LivrareCuReducere</code>	 <code>ArgumentOutOfRangeException</code> (vezi C1 mai jos)
C2 — <code>using System.Globalization</code> scos	
C1 — literalii <code>decimal</code> cu <code>m</code>	 ca stil — dar a introdus B2

Critice

B2 (regresie nouă) — formula din `LivrareStandard` e greșită

`ex1/Models/LivrareStandard.cs:9`

Când ai trecut la literalii `m`, ai copiat coeficienții de la Express în Standard:

```
return 5 + 1.0m * greutateKg + 0.25m * distantaKm; // sunt coeficienții Express
```

Enunțul cere pentru Standard `5 + 0.5·greutate + 0.1·distanța`. La rulare iese `Standard: 110.00` în loc de `57.00`, iar Standard și Express au acum practic aceeași formulă (diferă doar prin `5` vs `12`).

	Acum	Corect
Standard	<code>5 + 1.0m * g + 0.25m * d</code>	<code>5 + 0.5m * g + 0.1m * d</code>

Lecția (fix cea de la Q&A runda 1): un cleanup „doar cosmetic” a schimbat comportamentul. După ORICE refactor, chiar și unul care „nu schimbă logica”, re-rulează și compară cifrele. O zecimală schimbată din

greșeală nu dă eroare de compilare — trece tăcută până o vede cineva în output.

🟡 Importante

M2 (rămas, pct. 3) — cazul de eroare tot nu e demonstrat

ex1/Testare1.cs

Validarea greutății există în constructorul `Comanda`, dar `Testare1` n-o pune niciodată la treabă. Cerința 4 cerea o comandă cu greutate `0`, prinsă cu `try/catch (ArgumentException)` și mesajul afișat. Adaug-o la final.

🟢 Cleanups

- **C1** — la `ArgumentOutOfRangeException`, primul argument e `paramName`, nu mesajul. Acum mesajul tău ajunge afișat ca nume de parametru („Parameter 'Discount percent...'”). Idiomatic: `new ArgumentOutOfRangeException(nameof(procent), "Discount percent must be between 0 and 100")`. Subtilitatea care te-a prins: la `ArgumentException` primul arg e mesajul, la `ArgumentOutOfRangeException` e numele parametrului — de-aia s-a schimbat sensul când ai schimbat tipul.
- **C3 (rămas)** — `Nume => "Reducere"` tot ascunde ce strategie împachetează. Îl reluăm la lecția Decorator.

Q&A

Q1. (ex2) Scrie pe hârtie ce se întâmplă pas cu pas în `Dezaboneaza` dacă array-ul are 3 observatori și-i dai unul care NU e în listă. La ce index se oprește și de ce? Ce mărime ar fi trebuit să aibă `obsNou` ?

Q2. (ex1) Standard și Express dau acum 110 și 117 — aproape la fel. Dacă N-ai fi avut cifrele din enunț în față, ce te-ar fi făcut să bănuiești că Standard e greșit, doar uitându-te la output? (hint: ce rost mai are „Standard” dacă e cât „Express”?)

Q3. (ex2) `Aboneaza` și `Dezaboneaza` refac amândouă array-ul de la zero. De ce în C# nu poți pur și simplu „mări” un array existent, și ce structură din .NET face exact asta pentru tine (pe care noi am interzis-o intenționat aici, ca să vezi mecanismul)?



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro